UdaPlay

An AI Research Agent for the Video Game Industry

Technical Project Report: RAG, ChromaDB, Tool-Calling Agent and Long-Term Memory

Basis of the report: Completed project files and notebook_run_results (fourth live run)

Components: Udaplay_01 (offline RAG) and Udaplay_02 (agent)

Report date: October 1, 2026

Contents

1. Executive Summary
2. Problem Statement
3. Solution Statement
4. Architecture
5. Dataset
6. Part 1: RAG Pipeline
- 6.1 Vector Database
- 6.2 Retrieval and Evaluation
- 6.3 Semantic Search
- 6.4 Reusable Vector Store

7. Part 2: Agent Implementation
- 7.1 Tools
- 7.2 Agents
- 7.3 State Machines
- 7.4 Long Term Memory
- 7.5 Reporting System

8. Results
9. Value Statement
10. Limitations
11. Conclusion
12. References

1. Executive Summary

UdaPlay is an AI research agent for the video game industry. It answers questions about game releases, platforms, publishers and genres by combining retrieval over a local ChromaDB knowledge base with a live web search fallback, and it remembers what it learns for later sessions. The agent follows the retrieval-augmented generation pattern [1], uses an LLM judge to decide whether retrieved evidence is good enough [3], and runs as a stateful state machine whose behaviour can be inspected step by step [2, 5].

This report is based on the completed project files and on the fourth live run of both notebooks (notebook_run_results) . Part 1 builds a persistent vector database from 15 game files. Part 2 builds the agent, its three required tools, a conversation memory and a persistent long-term memory. Every claim below is traceable to a notebook output, a library file or a verified reference.

The report is organised as follows. Sections 2 to 4 define the problem, the solution and the architecture. Section 5 describes the dataset. Sections 6 and 7 document the two parts of the implementation. Section 8 presents the results, sections 9 and 10 discuss value and limitations, section 11 concludes, and section 12 lists the verified references. It is written for reviewers, engineers and stakeholders who need a complete picture of what was built, how it behaves and how far it can be trusted. All figures were produced from the project data and the run outputs.

The fourth run met every requirement in the project instructions. The main outcomes were:

- All 15 game documents were indexed in the persistent collection 'udaplay', and a freshly created manager reopened the same 15 documents from disk.
- Every factual turn followed the same order: retrieve_game first, evaluate_retrieval second, and the web only when the judge returned useful=False. The judge received all five retrieved documents each time.
- The Pokemon and Mario questions were answered from internal documents, while the Mortal Kombat X question, which is outside the dataset, triggered the web fallback.
- A follow-up question was answered from conversation context with no tool calls, which demonstrates short-term memory.
- Web findings were saved with their source URLs. A brand-new agent then answered the same question from memory with no Tavily call, cutting the turn from 16,420 to 6,906 tokens, a 58 percent reduction.

Across the four live runs, the tracked quality checks improved from 2 of 6 passing in the first run to 6 of 6 in the fourth. The decisive lesson was architectural: instructions placed only in the prompt were skipped by the model, so memory saving and recall were moved into code. The main residual risks are the very small dataset, a lenient LLM judge on superlative questions, and a retrieval threshold calibrated on only a few examples. The recommended next steps are a larger and richer dataset, an automated evaluation set, and deterministic routing of the internal-first workflow.

Table 1. Key facts at a glance

Item	Value (run 4)
Games indexed	15 JSON files in collection 'udaplay'
Embedding model	text-embedding-3-small (OpenAI)
Agent and judge model	gpt-4o-mini, temperature 0
Required tools	retrieve_game, evaluate_retrieval, game_web_search
Turns demonstrated	3 required queries, 1 follow-up, 1 cross-session query
Judge decisions	useful=True twice, useful=False twice, confidence 0.9 each time
Mortal Kombat X tokens	16,420 (web) versus 6,906 (memory)
Tavily calls on the repeat question	0

2. Problem Statement

Information about the video game industry is scattered across publisher sites, store pages, wikis, reviews and news. Analysts, product managers and journalists need fast answers to questions such as when a game was released, which platform it ran on, who published it and whether an older title still works on a new console. They also need to know where each answer came from.

Three common approaches each fail on their own. A bare large language model answers from parametric knowledge that is frozen at training time, cannot cite a source and can state wrong facts with confidence. A plain retrieval-augmented system [1] is grounded, but its knowledge base is closed. In this project the base holds only 15 games, so a question about Mortal Kombat X returns the nearest unrelated neighbours, with a best distance of 1.04, and a naive pipeline would still produce an answer from them. A web-search-only assistant reaches current facts but is noisy, costly, exposed to forum and user-generated content, and repeats identical searches with no continuity.

The cost of a wrong answer is not symmetric. A mistaken release year in a market report can mislead a decision, and a missing citation forces the reader to repeat the research. The problem is therefore not only to find an answer but to find one with evidence, to know how strong that evidence is, and to say so openly. Success is measured twice: the answer must be correct, and the path to it must be visible.

The project therefore asks for an agent that decides how to answer, rather than a fixed pipeline. It must answer from internal knowledge when that is sufficient, detect when it is not, search the web in that case, keep the conversation state, return structured and cited output, and store useful information for future use. Five concrete challenges follow from this brief.

The success criteria were taken directly from the project instructions: indexed data, three working tools, an ordered internal-first workflow, conversation state, cited answers to at least three example queries, and stored knowledge.

- Knowing when retrieval is good enough, because vector search always returns neighbours even when none are relevant.
- Grounding every answer in evidence and showing the sources and a confidence level.
- Keeping the context of a conversation so that follow-ups such as 'who published it?' work.
- Persisting knowledge across sessions without repeating paid web searches or storing duplicates.

- Making agent behaviour inspectable, so that a reviewer can see which tools ran and why.

A further engineering constraint came from the course setup. The supplied library code in the lib folder had to remain unchanged, and its vector store manager is in-memory only. The solution therefore had to extend the library in the notebooks rather than edit it.

Table 2. Problems, consequences and design responses

Problem	Consequence if ignored	Design response
Frozen LLM knowledge	Outdated or invented facts	Retrieve first, then use the web for gaps
Closed knowledge base	Irrelevant neighbours used as evidence	LLM judge with useful flag and confidence
Noisy web results	Forum or video content cited as fact	Domain exclusion and source rules in the prompt
No continuity	Follow-ups fail, searches repeat	Session memory and persistent long-term memory
Opaque agents	Behaviour cannot be audited	Per-turn report of path, tools and sources
Read-only library	Cannot add persistence by editing lib	Subclass the manager inside the notebooks

3. Solution Statement

UdaPlay is a hybrid research agent. It places a local semantic knowledge base first, a judge second, a web search third and a persistent memory behind both. The design follows the guidance that effective agents are simple, composable and transparent [5], and it uses a tool-calling loop in which reasoning and acting alternate [2].

The workflow for a factual question has five steps. The agent first resolves pronouns in follow-ups so that every tool call is self-contained. It then calls retrieve_game, which returns the five closest games with their distances. It next calls evaluate_retrieval, where a second LLM call, constrained by a Pydantic schema, returns a useful flag, an explanation and a confidence between 0 and 1. If the evidence is useful, the agent answers from the internal documents. If not, it calls game_web_search, which consults long-term memory before touching the network, and only then queries Tavily and saves the top results.

The roles are intentionally separated. Retrieval is deterministic and cheap, the judge is a narrow classification task with a typed output, and the web search is the only component that spends external credits. The agent loop coordinates these pieces but contains none of their logic, so each can be tested alone, as the notebook does when it calls the tools directly before the agent is built.

Every final answer uses the same four sections: Answer, Reasoning, Sources and Confidence. This makes outputs comparable across questions and gives a reader the audit trail directly. The notebooks add a reporting layer that prints the state machine path, each tool call with a one-line result summary, the final answer and the token count for each turn.

Several alternatives were considered. A fixed pipeline would be cheaper but could not skip the web when the internal answer is sufficient. A web-first design would be fresher but slower and costlier. Letting the model manage memory through tools was tried first and failed in practice, which led to the code-level design above. The chosen hybrid accepts one extra model call for the judge in exchange for far better control over when evidence is trusted.

The central design decision emerged from testing. In the first three live runs the agent ignored prompt instructions to search and register memory. The fix was to implement saving and recall inside game_web_search, so that the behaviour no longer depends on the model remembering an instruction. The principles that guided the solution are summarised below.

- Internal first: cheap, grounded and auditable knowledge is always tried before the web.
- Judge before trusting: retrieval quality is assessed explicitly, never assumed.
- Remember with provenance: every saved fact keeps its source URL and the question that produced it.
- Prefer code to prompts for invariants: anything that must always happen is done in code.
- Extend, do not edit: persistence is added by subclassing the supplied library classes.

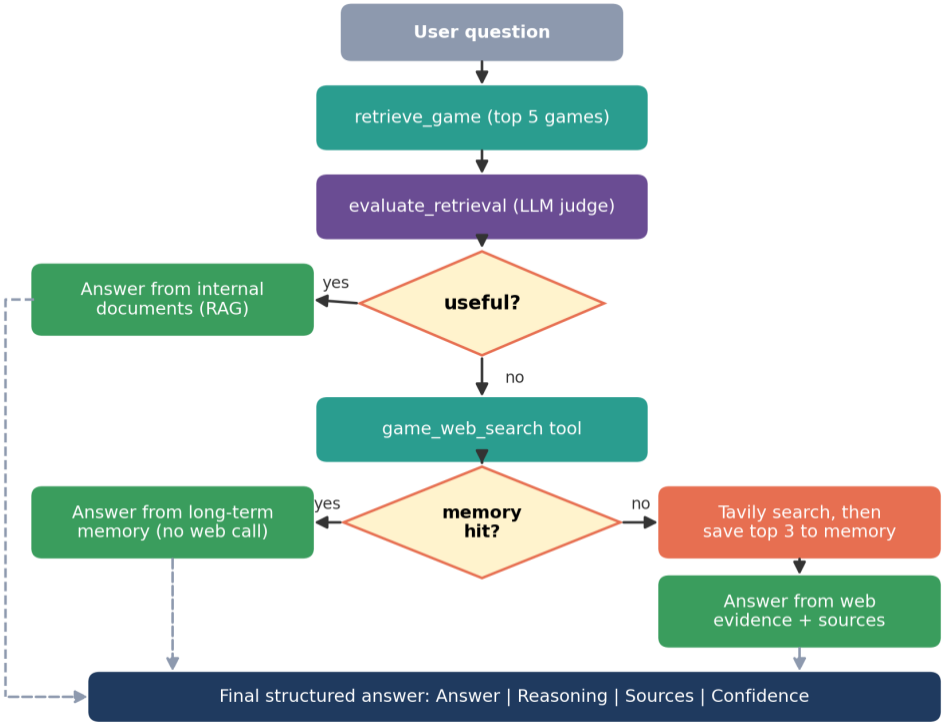


Figure 1. Decision workflow of the UdaPlay agent for one factual question.

Table 3. Requirement to solution mapping

Project requirement	Solution element	Where in the project
Process and store game data	Persistent Chroma collection, 15 documents	Notebook 1
Semantic search	store.search with optional metadata filter	Notebook 1
retrieve_game tool	Top-5 search with distances and source	Notebook 2

Project requirement	Solution element	Where in the project
evaluate_retrieval tool	LLM judge with EvaluationReport schema	Notebook 2
game_web_search tool	Tavily search with memory-first lookup	Notebook 2
Stateful agent	lib.Agent state machine with session memory	Notebook 2
Structured, cited output	Answer, Reasoning, Sources, Confidence	Notebook 2
Long-term memory	Persistent collection with source URLs	Notebook 2

4. Architecture

The system has five layers. The interaction layer is a Jupyter notebook whose ask() function sends a question to the agent and prints a report. The agent layer is the lib.Agent class, a state machine around gpt-4o-mini, with a ShortTermMemory object that stores one Run object per turn and session. The tool layer contains the three required tools. The knowledge layer is a persistent ChromaDB directory with two collections. The service layer consists of the OpenAI API for chat completions and embeddings, and the Tavily search API.

A typical request flows as follows. The question enters message_prep, which adds the system prompt on the first turn and appends the user message. The llm_processor step calls the model with the tool schemas. When the model returns tool calls, tool_executor runs them and returns the results to llm_processor. When it returns plain text, the machine terminates and the Run object, including all snapshots, is stored in session memory under the session identifier.

Responsibilities are separated by layer. The notebook owns presentation, the agent owns control flow, the tools own domain actions, and the stores own persistence. Because the tool functions use module-level objects such as the games store and the memory object, they can be called directly for testing, which the notebook does before the agent exists.

Two collections share one embedding function. The collection 'udaplay' holds the 15 game documents. The collection 'udaplay_long_term_memory' holds saved web findings with metadata for owner, namespace, timestamp, source URL and a normalised question key. Both live in ./chromadb, so they survive kernel restarts. Using the same embedding model in both notebooks is mandatory, because vectors from different models are not comparable.

Durability follows from the persistent client. A kernel restart loses the short-term session memory, which lives inside the Agent object, but not the stored vectors. The memory reset flag in the notebook exists only to make demonstrations reproducible. It is set to true by default, and setting it to false keeps memories between runs.

Configuration is deliberately small. API keys are read from config.env or .env. The OpenAI base URL defaults to the course gateway when OPENAI_BASE_URL is not set, and it is passed explicitly to the embedding function. Chroma telemetry is switched off through an environment variable so that notebook output stays clean.

Secrets are never printed. Keys are read from environment files and checked with assertions before any call is made. The OpenAI client reads the base URL from the environment, so neither a key nor a URL is hard-coded in the source.

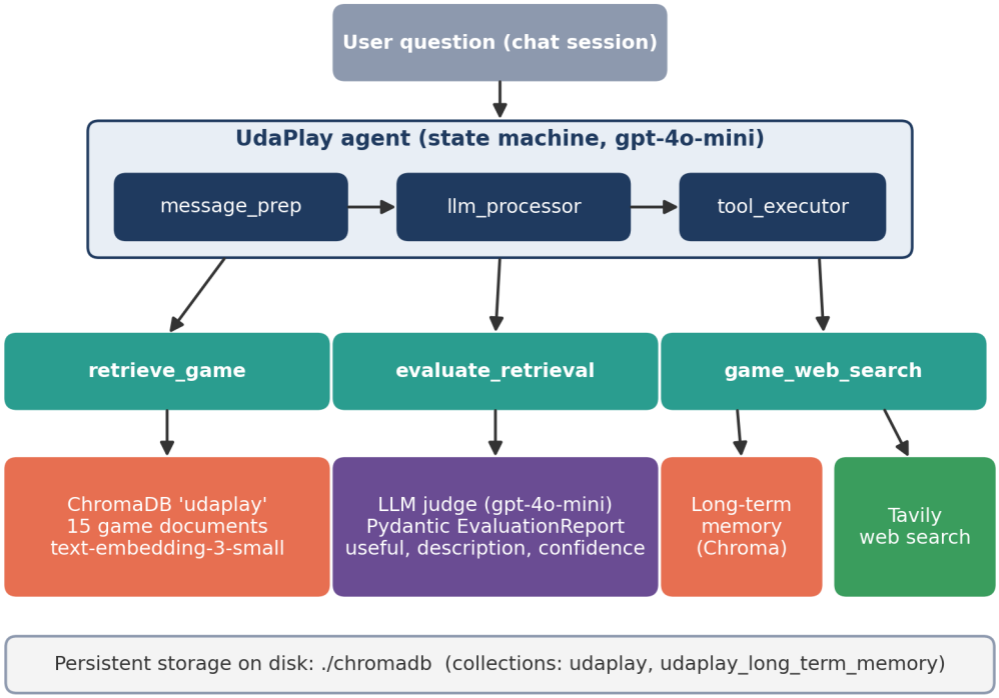


Figure 2. Component architecture of UdaPlay.

Table 4. Technology stack

Layer	Technology	Role
Language model	OpenAI gpt-4o-mini	Agent reasoning and retrieval judge
Embeddings	text-embedding-3-small, 1536 dimensions [9, 10]	Vectors for games and memories
Vector database	ChromaDB PersistentClient [6, 7]	Storage and similarity search
Web search	Tavily Python SDK [8]	Fallback evidence
Validation	Pydantic BaseModel	Typed judge output
Agent runtime	lib.Agent, StateMachine, ShortTermMemory	Loop, steps, sessions
Notebook runtime	Jupyter, Python 3.13 kernel	Execution and reporting

Table 5. Key configuration values

Setting	Value	Purpose
Collection name	udaplay	Game knowledge base

Setting	Value	Purpose
Memory collection	udaplay_long_term_memory	Saved web findings
Results per retrieval	5	Evidence given to the judge
Memory hit threshold	0.9 (L2 distance)	Semantic memory reuse
Web results saved	Top 3	Bounded memory growth
Tavily depth and size	advanced, 5 results	Relevance over speed
Excluded web domains	youtube, reddit, fandom, quora	Avoid user-generated content

5. Dataset

The dataset is a folder of 15 JSON files named 001 to 015. Each file describes one game with six fields: Name, Platform, Genre, Publisher, Description and YearOfRelease. The files are small and clean, which makes them suitable for demonstrating the pipeline, but they also define the boundary of what the agent can answer without the web.

The collection spans 33 years, from Super Mario World in 1990 to Marvel's Spider-Man 2 in 2023. It contains one game for each of 15 distinct platforms, from the PlayStation 1 to the PlayStation 5, from the Super Nintendo to the Nintendo Switch, and from the Xbox 360 to the Xbox Series X|S. Nine genres are present, led by Racing and Action-adventure with three games each. Nintendo published seven of the 15 games, Sony entities four, the Microsoft family three, and Rockstar Games one.

A review of the files shows consistent field names and types, with years stored as integers and one-sentence descriptions. Genre values are single labels except for Minecraft, which uses the combined value Sandbox, Survival. Publisher names vary inside a family, for example Sony Computer Entertainment and Sony Interactive Entertainment, so a question about Sony needs semantic matching and not an exact filter.

Each file becomes one document. The indexed text combines the platform, name, year, description, genre and publisher in a single string, so that a question about a publisher or genre can match the text and not only the description. The complete record is also stored as metadata, which allows exact filters such as Genre equal to Racing. The file name without extension becomes the document identifier, which makes re-indexing idempotent.

These properties shape retrieval behaviour. Short descriptions give each document a compact embedding, so a question that names a game matches strongly, with best distances between 0.65 and 0.79 in the run. A question about a game that is absent matches only weakly. The one-game-per-platform structure also explains why platform questions return a few ranked neighbours and not a complete list.

The dataset has clear limits. There is exactly one game per platform, so aggregate questions such as listing all PlayStation games cannot be answered completely. There are no developers, sales figures, ratings or release dates beyond the year. The newest entry is from 2023, and any question about later events must use the web. These limits are the reason the web fallback and long-term memory exist. For a reviewer the dataset is also a convenient test bed. It is small enough to read in full, yet varied enough to include racing, role-playing, platform, fighting, sports and party games from three console families, which exercises both exact filters and semantic matching.

Table 6. The 15 indexed games

Name	Platform	Genre	Publisher	Year
Gran Turismo	PlayStation 1	Racing	Sony Computer Entertainment	1997
Grand Theft Auto: San Andreas	PlayStation 2	Action-adventure	Rockstar Games	2004
Gran Turismo 5	PlayStation 3	Racing	Sony Computer Entertainment	2010
Marvel's Spider-Man	PlayStation 4	Action-adventure	Sony Interactive Entertainment	2018
Marvel's Spider-Man 2	PlayStation 5	Action-adventure	Sony Interactive Entertainment	2023
Pokémon Gold and Silver	Game Boy Color	Role-playing	Nintendo	1999
Pokémon Ruby and Sapphire	Game Boy Advance	Role-playing	Nintendo	2002
Super Mario World	Super Nintendo Entertainment System (SNES)	Platformer	Nintendo	1990
Super Mario 64	Nintendo 64	Platformer	Nintendo	1996
Super Smash Bros. Melee	GameCube	Fighting	Nintendo	2001
Wii Sports	Wii	Sports	Nintendo	2006
Mario Kart 8 Deluxe	Nintendo Switch	Racing	Nintendo	2017
Kinect Adventures!	Xbox 360	Party	Microsoft Game Studios	2010
Minecraft	Xbox One	Sandbox, Survival	Mojang Studios	2014
Halo Infinite	Xbox Series X|S	First-person shooter	Xbox Game Studios	2021

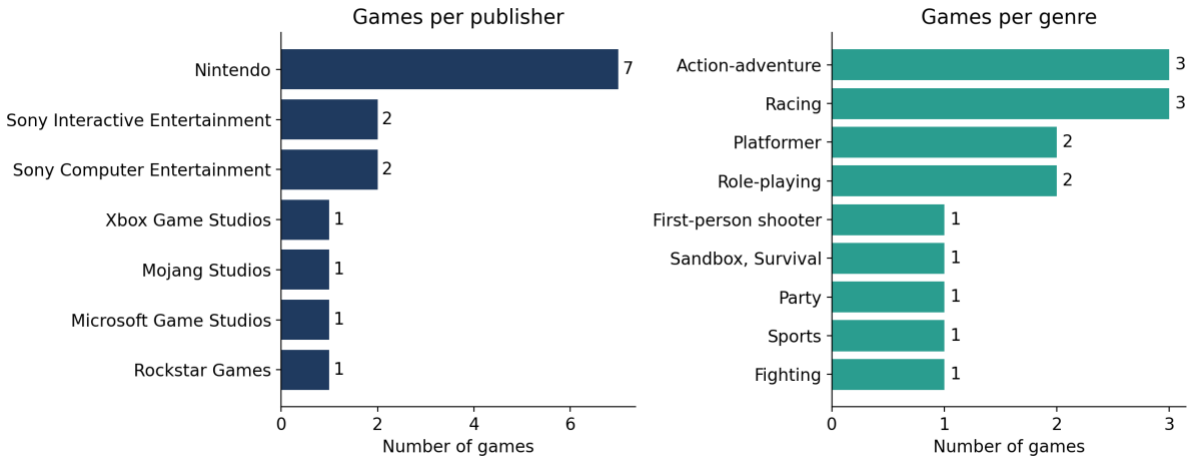


Figure 3. Distribution of the 15 games by publisher and by genre.

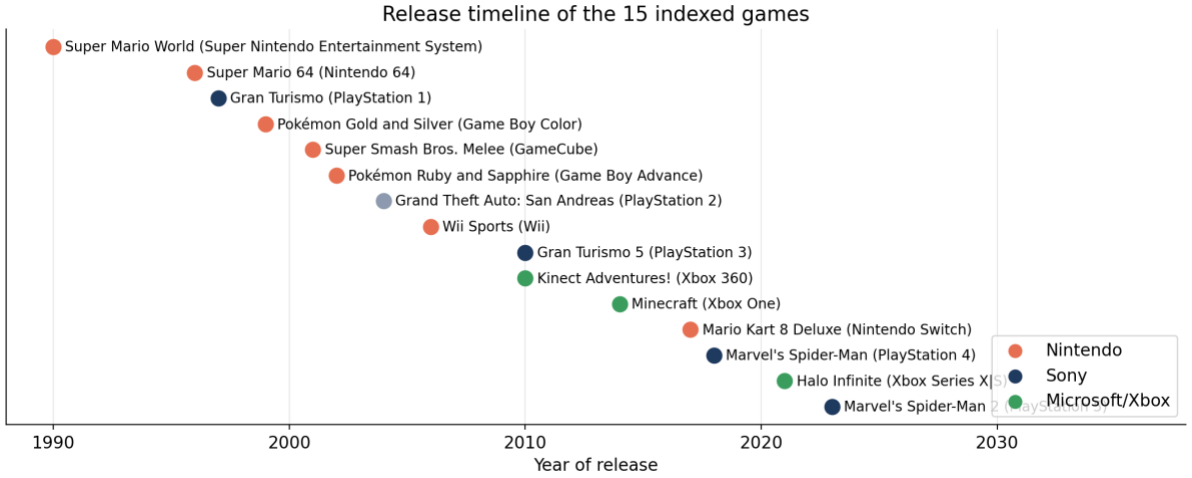


Figure 4. Release timeline of the indexed games, coloured by publisher family.

6. Part 1: RAG Pipeline

6.1 Vector Database

Notebook 1 creates a chromadb.PersistentClient at ./chromadb and a collection named 'udaplay'. The collection uses an OpenAI embedding function with the model text-embedding-3-small [9] and an explicit API base, so embeddings are created the same way on every run. Documents are added with upsert, not add, so running the notebook again updates records instead of failing on duplicate identifiers. The run showed 'Existing documents: 15' on the second execution and the same count after indexing, which confirms idempotence. The embedding vectors have 1536 dimensions for this model, and the collection stores the original text, the vectors and the metadata together, so a result can be returned without a second lookup.

```
store.upsert(game_documents)
assert store.count() == len(game_documents)
```

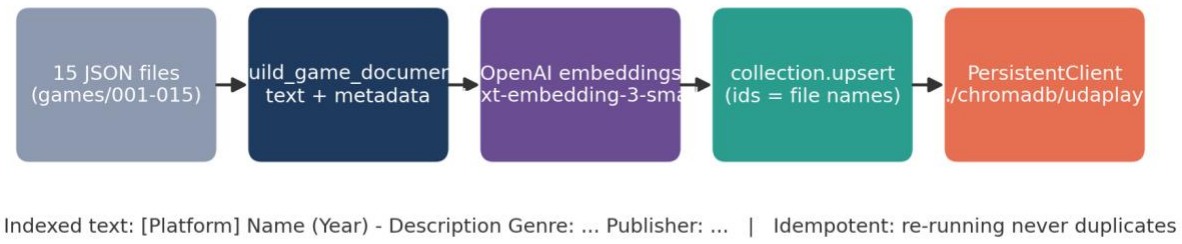


Figure 5. Ingestion pipeline from JSON files to the persistent collection.

6.2 Retrieval and Evaluation

Retrieval embeds the query with the same model and returns the nearest documents by Chroma's default L2 distance, were lower means closer. In run 4 the relevant questions had a best distance between 0.65 and 0.79, while the out-of-scope Mortal Kombat X question had a best distance of 1.04. Distance alone cannot prove irrelevance, because some relevant documents also sit near 1.0, so Part 2 adds an LLM judge that reads the documents and decides whether they answer the question. The retrieval tool returns the distance with every document and applies no cut-off, because a hard threshold would need calibration on far more data than 15 documents.

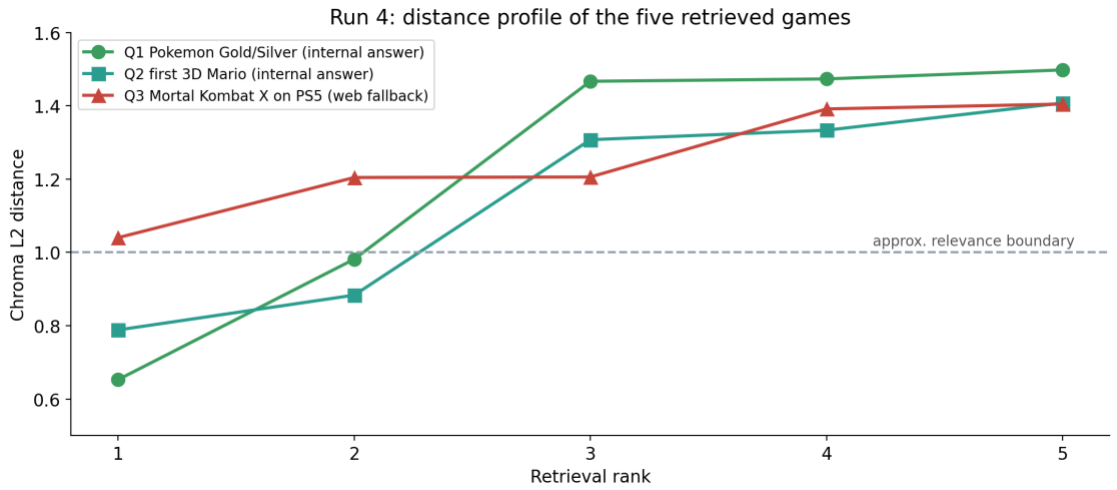


Figure 6. Distance profile of the five retrieved games for the three example queries in run 4.

6.3 Semantic Search

Four demonstration queries confirmed that semantic search works on meaning and on the enriched text. The publisher question ranked Halo Infinite first at 0.7221, far ahead of the next result at 1.2902. The blocky open world question ranked Minecraft first. A metadata-filtered query for fast cars with Genre equal to Racing returned only racing games. The broad question about PlayStation games returned PlayStation 3, 4 and 1 titles but not the PlayStation 2 or 5 games, which shows that semantic search ranks relevance and does not enumerate sets. Semantic search also accepts metadata filters, which gives exact control when a question names a genre or platform.

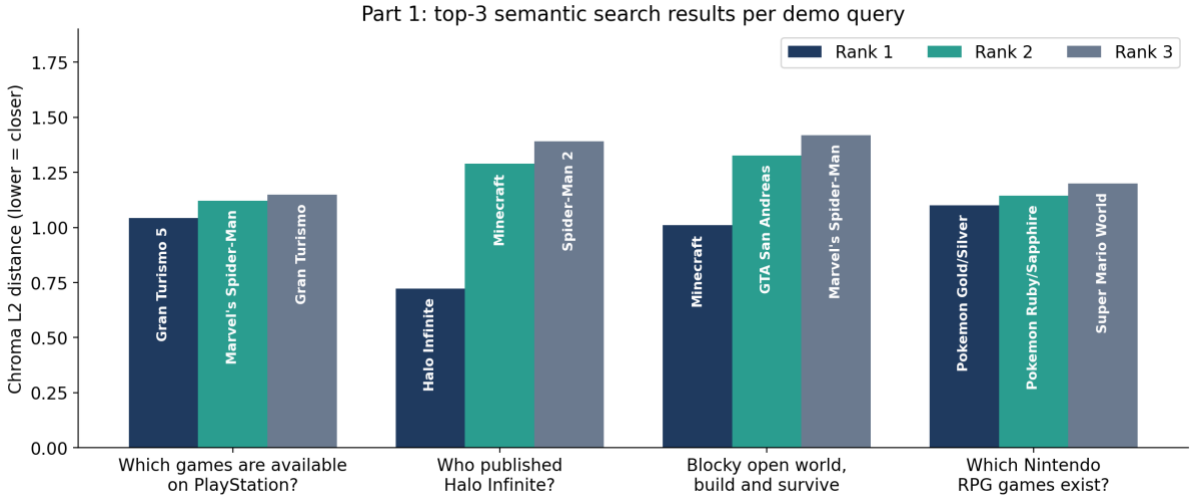


Figure 7. Top three semantic search results for the four demonstration queries.

Table 7. Top result per demonstration query

Query	Top result	Distance	Rank 2 gap
Which games are available on PlayStation?	Gran Turismo 5	1.0426	0.0788
Who published Halo Infinite?	Halo Infinite	0.7221	0.5681
Blocky open world, build and survive	Minecraft	1.0113	0.3151
Which Nintendo role-playing games exist?	Pokemon Gold and Silver	1.0994	0.0444

6.4 Reusable Vector Store

The supplied VectorStoreManager creates an in-memory client, which loses data when the kernel stops, and the library had to stay unchanged. The notebooks therefore define two small subclasses. GameVectorStore adds count, upsert and search helpers. PersistentVectorStoreManager keeps the same interface, namely get_store, get_or_create_store and delete_store, but writes to disk and fixes the embedding model and base URL. The same classes serve the game collection in Part 1 and both the game and memory collections in Part 2. A new manager instance reopened the 15 documents, which verified persistence. Using one manager class for every collection avoids configuration drift between the notebooks and makes the memory store behave exactly like the game store.

Table 8. Reusable vector store classes

Class or method	Base	Added behaviour
PersistentVectorStoreManager	VectorStoreManager	Disk persistence, chosen embedding model
get_or_create_store(name)	same	Returns a GameVectorStore
GameVectorStore.upsert(docs)	VectorStore	Idempotent insert or update
GameVectorStore.search(query)	VectorStore	Metadata plus distance as dictionaries
GameVectorStore.count()	VectorStore	Collection size check

7. Part 2: Agent Implementation

7.1 Tools

Each tool is a Python function wrapped by the @tool decorator, which builds a JSON schema from the signature and docstring. retrieve_game returns the five closest games with metadata, distance and a source label. evaluate_retrieval sends the question and documents to a second gpt-4o-mini call with a Pydantic response format. game_web_search checks long-term memory, then calls Tavily and saves the top three results. Docstrings are written for the model, because they are the tool descriptions it reads. Direct calls in the notebook, such as retrieving Pokemon Gold and Silver and then evaluating the result, proved each tool before the agent used it.

Table 9. Tool contract

Tool	Input	Output	Backend
retrieve_game	query	5 games with distance	ChromaDB
evaluate_retrieval	question, retrieved_docs	useful, description, confidence	LLM judge
game_web_search	question	Memory hits or web results	Memory, Tavily

```
class EvaluationReport(BaseModel):
    useful: bool
    description: str
    confidence: float  # 0 to 1
```

7.2 Agents

The agent is created by a build_agent() factory with gpt-4o-mini, temperature 0 and the three tools. A factory matters because the cross-session test needs a second, completely fresh agent. The system instructions define the workflow, require follow-up questions to be rewritten as self-contained queries, require all retrieved documents to be passed unchanged to the judge, forbid claims that tools do not support, and fix the four-part answer format. Keeping the instructions in one string means the same text is reviewed once and applied to every agent.

Table 10. Instruction rules and their purpose

Rule in the system prompt	Purpose
Resolve follow-ups before any tool call	Make retrieval queries self-contained
Pass all retrieved documents to the judge	Prevent self-filtering before evaluation
Use memory results only when they answer	Avoid irrelevant reuse
Never state unsupported facts	Reduce hallucination
Answer, Reasoning, Sources, Confidence	Auditable, uniform output

7.3 State Machines

The lib.Agent class builds a state machine with five steps. The entry point leads to message_prep, then to llm_processor. A transition function inspects the state: when tool calls are pending, the machine moves to tool_executor and then back to llm_processor, and otherwise it terminates. Each step produces a snapshot, so a Run object holds the full path. The reports show paths such as seven steps for a one-round internal answer and nine steps for the web fallback. The loop has no iteration limit, so a stopping rule would be needed before production use.

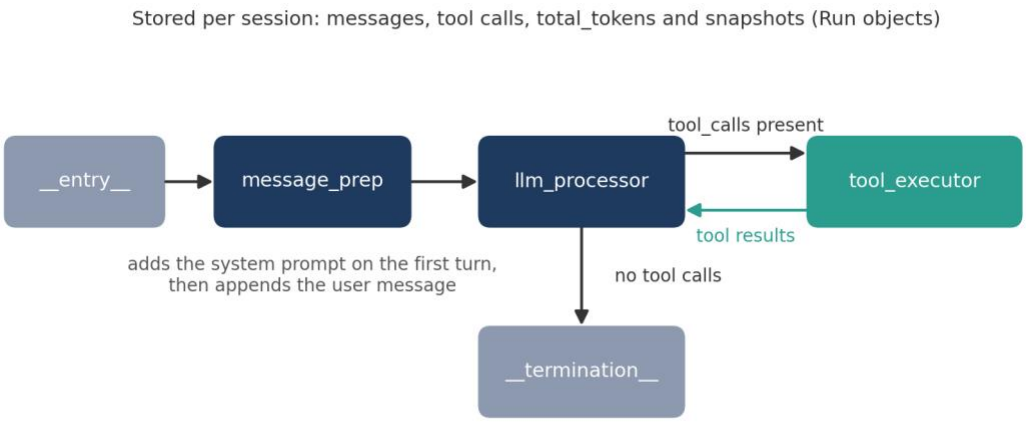


Figure 8. State machine of the agent loop.

7.4 Long Term Memory

Long-term memory uses the library's LongTermMemory class on top of the persistent manager, with owner udaplay-agent and namespace game-research. Saving and recall

are implemented in code inside game_web_search. After a Tavily search, the top three results are stored with source URL and a normalised question key, skipping duplicates. On the next call, an exact question-key match is tried first, then a semantic match with distance at or below 0.9. Because this is deterministic code, the model cannot skip it. The design reflects the idea of an external memory tier that holds knowledge outside the context window [4]. Entries also carry a timestamp, so time filters are possible in the library even though this project does not use them yet.

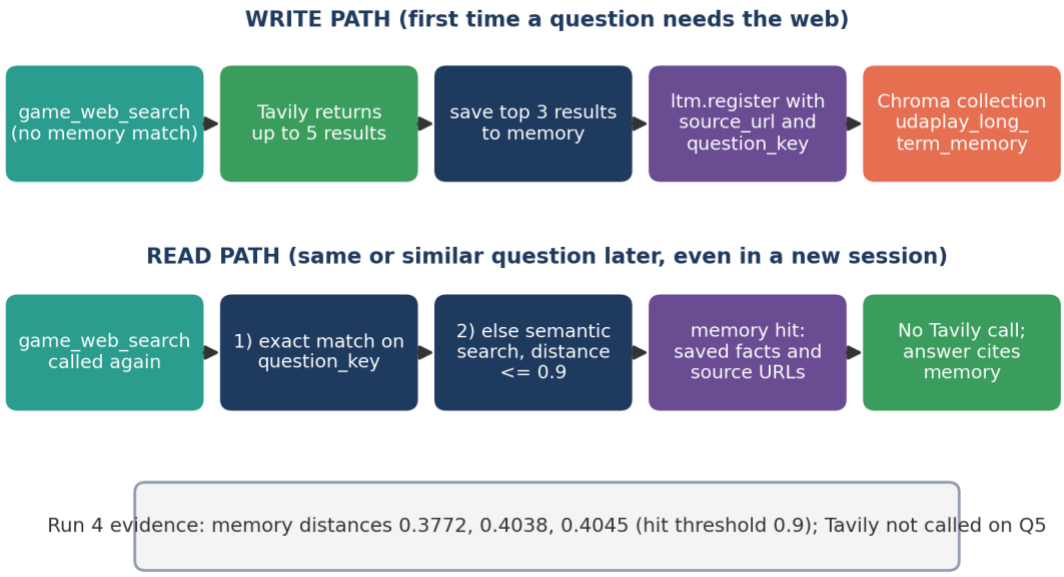


Figure 9. Write and read paths of long-term memory.

Table 11. Memories saved in run 4

Source	Stored content (abridged)	Reuse distance
store.playstation.com product page	Playable on PS5, some PS4 features may be absent	0.3772
pushsquare.com forum topic	Mortal Kombat X on PS5 snippet	0.4038
en.wikipedia.org Mortal Kombat X	Mortal Kombat XL released 2016 for PS4 and Xbox One	0.4045

7.5 Reporting System

The ask() function runs the agent with standard output redirected, because the state machine prints a log line per step, and then calls print_report. The report extracts only the current turn by locating the last user message, so earlier turns in the same session do not

repeat. It prints the question, the state machine path, each tool call with abbreviated arguments, a one-line result summary, the final answer and the token count.

Table 12. Report anatomy

Section	Content	Purpose
QUESTION	The user question	Context
STATE MACHINE PATH	Step identifiers in order	Control-flow proof
TOOL USAGE	Calls, arguments, result summaries	Audit of decisions
FINAL ANSWER	Answer, Reasoning, Sources, Confidence	Deliverable
Token count	Total tokens in the turn	Cost visibility

8. Results

Notebook 1 produced the expected knowledge base. It indexed 15 documents, retrieved Pokemon Gold and Silver as the top hit for a Pokemon question, and reopened the collection from disk with the same count. Notebook 2 ran without errors and produced five reported turns, summarised in the table below.

The first two questions were answered from internal documents in two tool rounds each. Pokemon Gold and Silver was reported as a 1999 Game Boy Color release at distance 0.6528 and judge confidence 0.9. Super Mario 64 was identified as the Nintendo 64 platformer of 1996 at distance 0.7884. The third question, Mortal Kombat X on PlayStation 5, produced a best internal distance of 1.0402, a judge verdict of useful=False and a web search. The answer stated that the game is playable on PlayStation 5 but was originally released for PlayStation 4, citing Wikipedia and the PlayStation Store.

The distance profile in Figure 6 explains the judge's decisions. For Q1 and Q2 the first one or two documents are clearly separated from the rest, while for Q3 all five distances lie between 1.04 and 1.40 with no leader. A human reader and the judge can both see the difference, and the judge reported confidence 0.9 in all four judged turns.

The fourth turn, a follow-up about the first game, used no tools and correctly answered Nintendo and Role-playing from context. The fifth turn used a new agent and a new session. The judge again returned useful=False, but game_web_search returned three memory hits at distances between 0.3772 and 0.4045 with no Tavily call, and the turn needed 6,906 tokens instead of 16,420.

Cost followed the path taken. The internal answers used 4,501 and 7,749 tokens, the web turn 16,420 and the memory turn 6,906. The web turn was the most expensive because search content was added to the prompt. In total the five turns made ten tool calls.

The iteration history shows how the solution matured. The first run missed memory reuse and gave the judge a single document. The second run reused memory but saved duplicates without URLs. The third run did not save memory at all because the model skipped the tool. The fourth run passed all six tracked checks after saving and recall moved into code. Two cosmetic issues remain in the answers: the Mario answer says the documents clearly identified a first, though the dataset only says 3D platformer, and the memory answer says a web search was conducted.

All answers agreed with the retrieved evidence. The Mortal Kombat X answer is also consistent with the cited store page, which states that the game is playable on PlayStation 5 and that some features available on PlayStation 4 may be absent.

Table 13. Results of the five reported turns

Turn	Tools called	Key evidence	Outcome	Tokens
Q1 Pokemon Gold and Silver release	retrieve, evaluate	Distance 0.6528, useful=True, 0.9	1999, internal sources	4,501
Q2 First 3D Mario platformer	retrieve, evaluate	Distance 0.7884, useful=True, 0.9	Super Mario 64, 1996	7,749
Q3 Mortal Kombat X on PS5	retrieve, evaluate, web	Distance 1.0402, useful=False, 0.9	Playable via PS4 version, web sources	16,420
Q4 Follow-up on first game	none	Answered from context	Nintendo, Role-playing	5,594
Q5 Same question, new agent	retrieve, evaluate, web (memory)	3 hits, 0.3772 to 0.4045	Same answer, no Tavily call	6,906

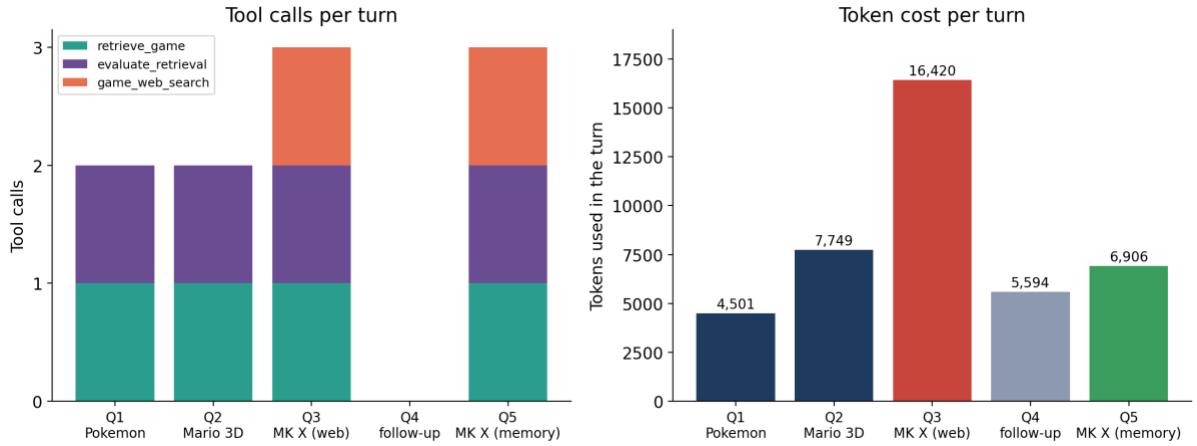


Figure 10. Tool calls and token cost for each reported turn.

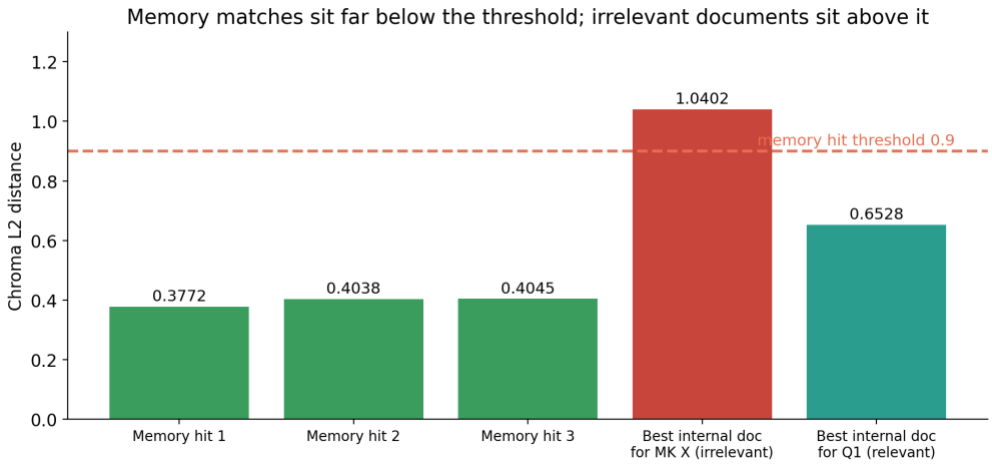


Figure 11. Memory hit distances compared with the threshold and with irrelevant retrieval.

Table 14. Iteration log across the four live runs

Run	Change before the run	Outcome
1	Persistent manager, reporting, memory tools in the prompt	Core met, memory never read, judge saw one document
2	Memory lookup in code, all documents to judge	Memory hit at 0.31, but duplicate and URL-less memory
3	Mandatory source URL, dedupe, domain exclusion	Agent skipped saving, no reuse
4	Saving and recall inside game_web_search	All checks passed, memory reused

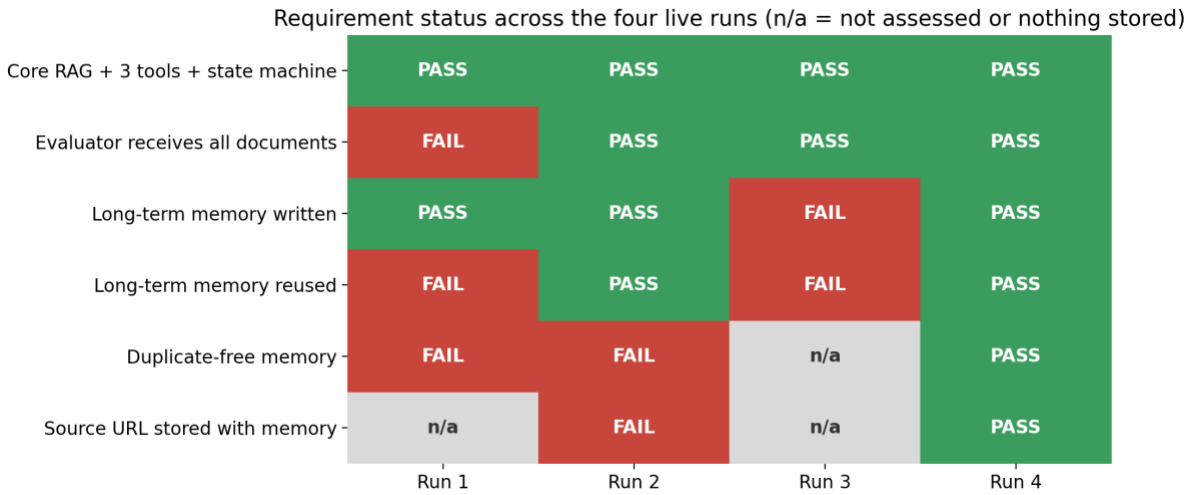


Figure 12. Status of tracked requirements across the four live runs.

Table 15. Requirement verification in run 4

Requirement	Evidence	Status
Persistent vector database	15 documents, reopened by a new manager	Met
Semantic search shown	Four queries plus metadata filter	Met
Three required tools	All three called in the reports	Met
Internal first, then evaluate, then web	Order in Q1, Q2, Q3 and Q5	Met
Stateful agent	Q4 answered with no tools	Met
Three or more queries with reasoning and sources	Five reported turns	Met
Information stored for future use	Three memories with URLs, reused in Q5	Met

9. Value Statement

UdaPlay turns a closed knowledge base into a research assistant that knows its own limits. For a team that tracks games, the main value is trustworthy speed: a question is answered in seconds, the answer names its sources, and the agent says when its evidence is thin. The judge step prevents the most damaging failure of simple retrieval systems, which is a confident answer built from irrelevant documents.

The explicit judge also lowers risk. When evidence is missing, the agent says so and changes its source instead of stretching weak material. In the run the judge rejected five unrelated PlayStation and Xbox documents for the Mortal Kombat X question, which is exactly the moment where an unguarded system would have guessed.

The agent also controls cost. The first Mortal Kombat X answer used 16,420 tokens and an advanced Tavily search, while the repeat used 6,906 tokens and no search, a 58 percent token reduction and zero additional search credits. In a team setting, where colleagues ask overlapping questions, saved findings with source URLs compound into a shared, auditable research memory. Internal answers were cheapest, averaging 6,125 tokens for the two internal questions.

Compared with the alternatives, the hybrid gives each approach a role. A plain chatbot would be quicker to build but unverifiable, a retrieval system alone would be precise on its own data but silent about everything else, and a search-only tool would be current but expensive and noisy. UdaPlay keeps the strengths of each and limits their weaknesses, which is the practical meaning of the value it delivers.

The design is valuable beyond games. The same pattern of internal retrieval, judged sufficiency, web fallback and persistent memory applies to product catalogues, policy libraries and competitor tracking. Because tools, memory and reporting are separate pieces, each can be replaced without redesigning the rest. For learners and reviewers, the per-turn report makes agent behaviour visible, which supports debugging, grading and trust.

The architecture scales by replacement and not by rewriting. A larger dataset needs only more files and the same indexing code, a stronger judge needs only a different model name, and a different search provider needs only a new tool body. For training and assessment, the project turns agent concepts into artefacts that can be inspected: a schema, a state machine, a memory and a report.

Finally, the project documents an engineering lesson with real value: invariants must live in code. Three live runs showed that a capable model can ignore a prompt rule, while a

code-level implementation passed the fourth run completely. Teams adopting agents can reuse this lesson to decide which behaviours to prompt and which to enforce.

Table 16. Value by stakeholder

Stakeholder	Benefit	Evidence in this project
Industry analysts	Fast, cited answers	Sources and confidence in every answer
Product managers	Lower research cost	58 percent fewer tokens on repeats
Engineers	Reusable, inspectable pattern	Reports, factory, subclassed manager
Reviewers and learners	Visible agent behaviour	State machine path and tool trace
Data owners	Provenance and control	Source URLs stored with every memory

Table 17. Quantified indicators from run 4

Indicator	Value	Comment
Token reduction on repeated question	58 percent	16,420 to 6,906
Tavily calls on repeated question	0	Memory hit
Average tokens, internal answers	6,125	Q1 and Q2
Judge confidence on judged turns	0.9	All four judged turns
Tracked checks passed	6 of 6	Up from 2 of 6

10. Limitations

The results are encouraging but they come from a small and controlled setting, and several limits should be stated plainly. The first is scale. The dataset has only 15 games, one per platform, and the live evidence is one run of five turns. There is no statistical analysis, no labelled evaluation set and no measure of recall or precision beyond inspection. Results at temperature 0 are not guaranteed to repeat exactly.

The second group concerns judgement. The LLM judge is useful but lenient: for the question about the first 3D Mario game, it accepted a document that only said 3D platformer, and the agent then wrote that the documents clearly identified a first. LLM judges are known to show bias and limited reasoning [3]. The distance boundary near 1.0 and the memory threshold of 0.9 were calibrated on a handful of examples and should be re-tuned on a larger set.

The third group concerns control. The internal-first order is still enforced by the prompt, and the model followed it in every turn, but nothing in code guarantees it. Memory stores raw search snippets instead of distilled facts, has no expiry or conflict handling, and can therefore serve stale information. The domain exclusion list is coarse, and a forum thread was still returned and saved in run 4, although the answer did not use it.

Privacy is a further concern. Search queries and saved snippets leave the local machine when they are sent to the model and search providers, so private questions should not be asked in this configuration. Saved memories are stored unencrypted on disk. The numbers in this report also describe one configuration on one dataset, and token counts depend on what the search provider returns on the day, so the 58 percent reduction is indicative and not a guaranteed figure.

The remaining limits are practical. The vector store classes are copied into both notebooks. The web path depends on a paid third-party API, and an advanced search is slower and costlier than a basic one. Secrets are read from local files, which is acceptable for a course project but not for production. The table below lists each limit with its evidence and a proposed next step.

The workflow depends on external services as well. An outage or rate limit at the model or search provider stops the web path, and the notebooks contain no retry or fallback logic.

A last limitation concerns generalisation. The domain is narrow, the questions were chosen by the activity authors, and there were no adversarial inputs such as ambiguous titles, misspelled names or questions in other languages. Robustness to such inputs remains untested.

Table 18. Limitations, evidence and next steps

Limitation	Evidence	Next step
Tiny dataset	15 games, one per platform	Expand data, add developers and ratings
Single-run evidence	One run of five turns	Repeat runs, build a labelled test set
Lenient judge	Q2 accepted without a stated first	Stricter rubric, second judge, human audit
Prompt-enforced order	No code guarantee of internal first	Make each tool a state machine node
Uncalibrated thresholds	Boundary 1.0 and 0.9 from few examples	Tune on labelled queries, consider cosine
Raw snippets in memory	Entries are search excerpts	Distil facts, add timestamps and expiry
Residual user content	Forum thread saved in run 4	Source allow-list, relevance score filter
Duplicated notebook code	Classes copied into two notebooks	Move to a shared module if lib may change
Cost and latency	16,420 tokens for the web turn	Basic search depth first, caching, batching

11. Conclusion

UdaPlay meets the objective of the project. It indexes a game dataset in a persistent vector database, exposes retrieval, evaluation and web search as tools, runs them inside a stateful state machine, answers with structured and cited output, and stores useful findings for later use. The fourth live run verified each requirement with evidence from notebook outputs, and a fresh agent reused earlier research without a second web search.

In numbers, the fourth run indexed 15 documents, answered five turns with ten tool calls, received four judge verdicts with confidence 0.9, and reused three saved memories with source URLs. The tracked checks rose from 2 of 6 in the first run to 6 of 6.

The project also shows how an agent should be built. A closed knowledge base needs an explicit sufficiency check, because vector search always returns neighbours. A web fallback needs source discipline. Memory needs provenance and deduplication. Above all, behaviour that must always happen belongs in code, because the live runs showed that prompt instructions alone were skipped. The final design keeps the model responsible for judgement and language, and keeps determinism where determinism is required.

Three lessons stand out. First, test agents with live runs, because only live runs reveal that a model skips an instruction. Second, show the path: the per-turn report made the memory failure in the third run visible at once. Third, design for reuse: the same manager class served both the games and the memory.

The work was done within a strict boundary. The supplied library files were not modified, and persistence and memory were added through subclasses and notebook-level helpers. This keeps the solution compatible with the course materials while still meeting the requirement for a persistent, reusable store.

For reviewers the evidence is complete and reproducible. The notebooks run from top to bottom, the outputs are saved, and every number in this report can be traced to one of them. The remaining weaknesses are listed openly in section 10, together with a concrete next step for each.

The recommended next steps are, in order of value:

- Build a labelled evaluation set of at least 50 questions and report precision, recall and judge agreement.
- Enrich and enlarge the dataset with developers, ratings and several games per platform.

- Promote the internal-first workflow to explicit state machine nodes so that ordering is guaranteed.
- Store distilled facts with timestamps and add expiry and conflict resolution to memory.
- Replace the fixed distance thresholds with values tuned on the evaluation set.

With these additions, the same architecture can grow from a course project into a dependable research tool for any domain where answers must be fast, grounded and traceable.

12. References

- [1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., and others (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. https://arxiv.org/abs/2005.11401
- [2] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. https://arxiv.org/abs/2210.03629
- [3] Zheng, L., Chiang, W.-L., Sheng, Y., and others (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS 2023 Datasets and Benchmarks Track. https://arxiv.org/abs/2306.05685
- [4] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560. https://arxiv.org/abs/2310.08560
- [5] Anthropic (2024). Building effective agents. Published 19 December 2024. https://www.anthropic.com/engineering/building-effective-agents
- [6] Chroma (n.d.). Collections, Chroma documentation. https://docs.trychroma.com/docs/collections
- [7] Chroma (n.d.). Python client reference: EphemeralClient and PersistentClient. https://docs.trychroma.com/reference/python/client
- [8] Tavily (n.d.). Python SDK quickstart. https://docs.tavily.com/sdk
- [9] OpenAI (n.d.). Vector embeddings guide. https://developers.openai.com/api/docs/guides/embeddings
- [10] OpenAI (n.d.). Embeddings API reference. https://developers.openai.com/api/reference/resources/embeddings